PhD RESEARCH PROPOSAL

When can we trust AI-generated tests? Uncertainty quantification for risk-based testing of autonomous CPS

Za Vinh Le



Write tests manually does not scale



- **Every release:** tests are rewritten by hand
- **AI can now read a spec and write tests [27]**

Goal

One tool: from the requirements document to a bug report, with AI writing the tests

[27] J. Wang et al., “Software testing with large language models: Survey, landscape, and vision,” IEEE TSE, 2024.

QA-Flow: from a requirements document to a bug report



Trust: hand-written rules

- Evidence required
- Uncertainty is flagged
- Fault-injection gate: every passing test is re-run on broken builds

Demo application: A shopping website

- Inputs: Software Requirement Specifications (SRS)
- Web, API, Security Testing
- LLM: Chat GPT

It finds real bugs — but 6 of its 18 FAILs are false alarms

34

requirements

from the document

62

tests

web 23 · API 26 · security 13

31 / 18 / 13

PASS / FAIL / REVIEW

decided by the tool

6 FAILs are mistakes in the AI's own tests

- Discount test used an empty cart → the real discount bug was missed
- Privacy tests logged in the wrong way → the real data leak was missed
- They passed every trust rule
- The report shows them as real bugs

What PASS, FAIL, REVIEW and false alarm mean

PASS

The check held, and the test fails on a broken shops

It can catch bugs.

Example: Log in with the right password → the account page opens.

31 in Demo 1

FAIL

The check did not hold

The tool reports a bug in the shop.

Example: SAVE10 gives €79.00 instead of €80.10.

18 in Demo 1

REVIEW

The tool does not trust the test

A person decides.

Example: A test that passes even on a broken shop: it cannot catch anything.

13 in Demo 1

FALSE ALARM

A FAIL where the mistake is in the AI's test, not in the shop. The tool cannot tell.

Example: The SAVE10 test that used an empty cart.

6 of the 18 FAILs

The tool decides PASS / FAIL / REVIEW by itself. False alarms were found only afterwards, by re-running every FAIL on a shop with the planted bugs removed.

Inside one wrong test: the SAVE10 discount code

Requirement: "SAVE10 gives 10% off." Planted bug in the shop: SAVE10 takes off €10 instead.

What a correct test would do

1. Put shoes (€89.00) in the cart
2. Enter the code SAVE10
3. Expect €80.10 → sees €79.00

FAIL → the real discount bug is found

What the AI's test did

1. Opened the cart directly — it was empty
2. Never reached the discount
3. Saw €0.00 → stopped

FAIL → reported as a bug, but the bug is in the test

Same for privacy: the test logged in the wrong way, so the real data leak was never checked.

The rules check the form of a test, not its meaning

Trust rule in QA-Flow	What it checks	The wrong SAVE10 test
Schema check	The AI's answer has the right format	✓ passed
Grounding check	Pages and fields exist; the quote matches the document	✓ passed
NEEDS REVIEW flag	Tests with no source go to a person	✓ not flagged
Fault-injection gate	A passing test must fail on a broken shop	– skipped (the test failed)

No rule asks: does this test really test the requirement?

Three ways a test can be wrong:

Wrong scenario | **Wrong check** | **Catches no bug**

Why it matters for self-driving cars

- **AI already helps build them** [8, 10]
- AI now writes their tests too [27]
- **A wrong test means** false safety

[8] J. Wu et al., “Reality bites: Assessing the realism of driving scenarios with large language models,” Proc. IEEE/ACM FORGE, 2024.
[10] C. Lu et al., “Foundation models for software engineering of cyber-physical systems: The road ahead,” ACM TOSEM, 2026.
[27] J. Wang et al., “Software testing with large language models: Survey, landscape, and vision,” IEEE TSE, 2024.

When can we trust an AI-generated test?

My idea Ask the AI for the same test several times. Tests agree → trust more. Tests differ → a person checks. Known for chatbots [24], new for tests.

Uncertainty in CPS testing

- Uncertainty-wise test generation for cyber-physical systems [1]

Testing AI-enabled systems

- Falsification: searching for failures of AI-enabled control systems [14]

Industrial test oracles

- Machine learning as a test oracle for elevator software [16]

Gap Uncertainty is studied for systems, robots and chatbots — not yet for AI that writes tests.

[1] M. Zhang et al., “Uncertainty-wise test case generation and minimization for cyber-physical systems,” J. Syst. Softw., 2019.
[14] Z. Zhang et al., “FalsifAI: Falsification of AI-enabled hybrid control systems guided by time-aware coverage criteria,” IEEE TSE, 2023.
[16] A. Arrieta et al., “Using machine learning to build test oracles: An industrial case study on elevators dispatching algorithms,” Proc. IEEE/ACM AST, 2021.
[24] S. Farquhar et al., “Detecting hallucinations in large language models using semantic entropy,” Nature, 2024.

The four PhD objectives

O1

New UQ metrics

Which signals show that an AI-written test cannot be trusted?

O2

Risk & trustworthiness

How should these signals decide accept / review / reject, given the safety risk?

O3

Uncertainty vs. quality

How do they relate to correctness, bugs found and cost?

O4

Future AI models

Do they still work for newer AI and quantum ML models?

[30] InnoGuard: Hybrid and Generative Intelligence for Trustworthy Autonomous Cyber-Physical Systems, Horizon Europe MSCA Doctoral Network, GA No. 101169233, 2024–2028. https://cordis.europa.eu/project/id/101169233

From Demo 1's gaps to the PhD plan

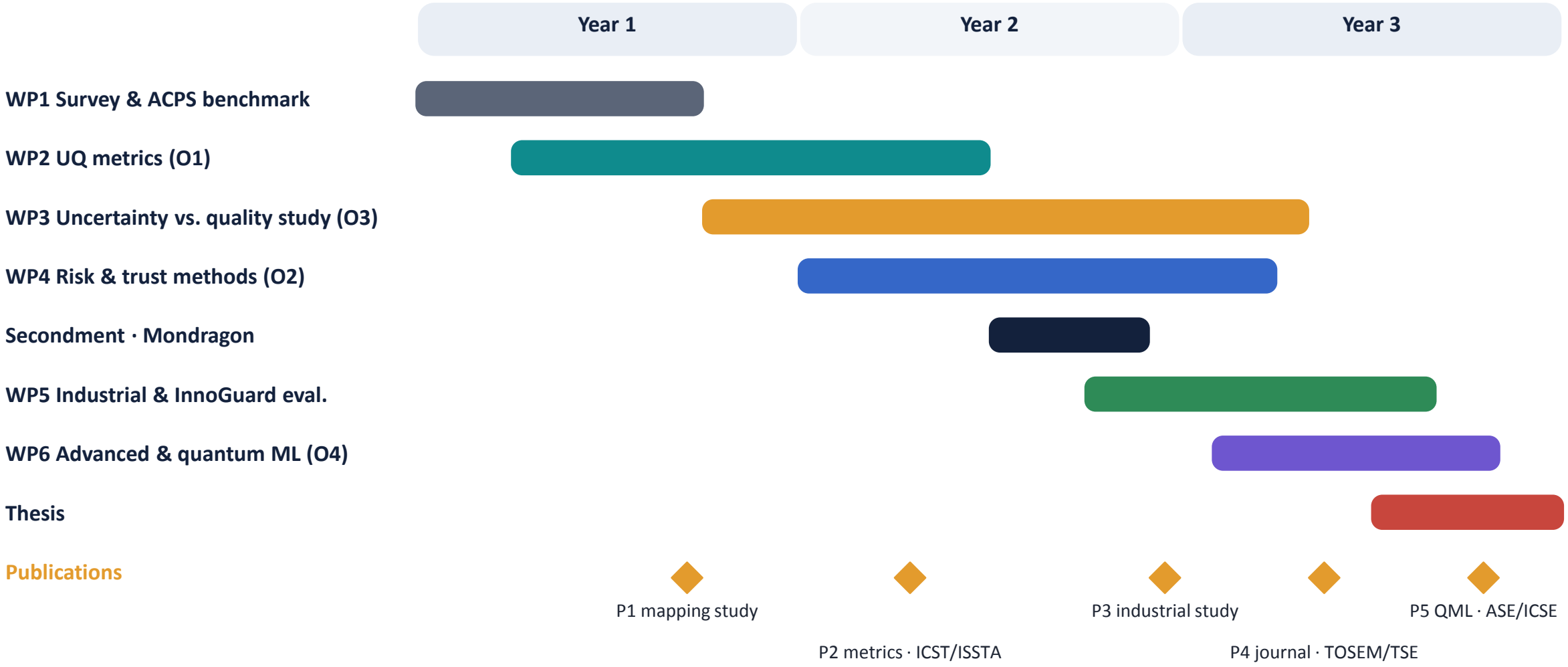
From	What Demo 1 cannot do	What the PhD will do	Obj.
Demo 1	Cannot tell a wrong test from a real bug (6 of 18)	Signals that show when a test is untrustworthy	O1
Demo 1	Every FAIL looks the same → a person checks all 18	Accept / review / reject based on risk	O2
Demo 1	Wrong tests hid 2 real bugs	Measure how trust relates to correctness and cost	O3
Next	One web shop, one AI model	Self-driving simulators, industrial cases, InnoGuard platform, new AI models	O4

Research questions and how I will answer them

RQ1	**Which signals best predict a wrong AI-written test?**	Compare signals (agreement, behaviour, model probabilities) on many generated tests
RQ2	**How can these signals support safe accept / review decisions?**	Risk = signal × how severe a failure is; measure the review effort saved
RQ3	**How do they relate to bugs found and cost?**	Large study: correctness, bugs found, run time, AI cost
RQ4	**Do they transfer to new and quantum AI models?**	Re-apply them to newer models and to quantum ML [20, 21]

[20] X. Wang et al., “Quantum machine learning-based test oracle for autonomous mobile robots,” Proc. IEEE/ACM ASE, 2025.
[21] X. Wang et al., “Application of quantum extreme learning machines for QoS prediction of elevators' software in an industrial context,” Companion Proc. ACM FSE, 2024.

36-month plan with secondment and publications



Expected contributions to InnoGuard

Metrics — Signals that tell when an AI-written test can be trusted

Methods — Accept / review / reject with safety-aware thresholds

Evidence — How trust relates to correctness, bugs found and cost

Open science — QA-Flow, an open benchmark and replication packages

Dissemination & network

- Venues: ICSE, FSE, ASE, ISSTA, ICST; TOSEM, TSE, EMSE
- Joint papers across the network, building on the team's foundation-model road maps [10, 11]
- Secondment at Mondragon: industrial CPS case and oracle expertise
- Training, summer schools and outreach of the doctoral network

[10] C. Lu et al., “Foundation models for software engineering of cyber-physical systems: The road ahead,” ACM TOSEM, 2026.
[11] S. Ali et al., “Foundation models for the digital twins creation of cyber-physical systems,” Proc. ISoLA 2024, 2025.

My Background

2019–23

B.S. Electrical & Electronics Eng.

Research: Autonomous Driving Car Using Reinforcement Learning

2022–24

Software Test Enginer Bosch @ Vietnam

Automotive ECU Testing for ADAS system

2024–26

M.S. Computer Science · KNU

Research: Software Testing and Verification

2026–

Software Test Engineer BNF @ Korea

Web/API/system testing LLM, RAG & MCP-based test generation

Thesis Towards Efficient Function Abstraction via Active Learning for Property Falsification (advisor: Prof. Yunja Choi)

Papers Test generation for property checking (KCC 2025); stateful system, stateless techniques: test generation for controller software (KCSE 2026), with S. Hong and Y. Choi

What I bring, and why this team

Testing cyber-physical systems

Automotive ECU testing (CAN, CAPL, ETAS INCA); robotics (ROS/ROS2) and embedded domains

Verification research

Falsification, function abstraction via active learning, property-based testing — close to robustness-guided falsification [14]

LLMs for quality engineering

Built QA-Flow end to end: RAG, LLM agents, MCP-based workflows, Playwright

Python & empirical studies

Controlled experiments on test generators (KCC'25, KCSE'26); ground-truth check of QA-Flow in this talk

Why this team

- Uncertainty-wise CPS testing (Simula)
- ADS testing & falsification (NII)
- Industrial CPS oracles & elevators (Mondragon)
- My work sits exactly where the three meet

[14] Z. Zhang et al., “FalsifAI: Falsification of AI-enabled hybrid control systems guided by time-aware coverage criteria,” IEEE TSE, 2023.

Thank you



Backup · QA-Flow demo: planted defects, faults and gates

Planted defect (ShopLite)	Type	Found by test(s)
SAVE10 subtracts €10 instead of 10 %	web	not found
Unknown product → 500 instead of 404	API	TC-19.1
Quantity 0 / negative accepted	API	TC-09.2, TC-22.2, TC-24.2, TC-24.3, TC-25.1
No Content-Security-Policy header	security	TC-06.3, TC-29.1, TC-29.2, TC-30.1
Reflected XSS in search	security	TC-06.3, TC-30.1
Login reveals registered emails	security	TC-31.1, TC-32.1
IDOR: other users' orders readable	security	not found

Run details

- 34 requirements, 62 tests (51 LLM-designed, 11 rule-derived)
- Schema gate: 55 agent replies, 2 rejected, 0 repaired
- Grounding gate: 0 tests ungrounded → NEEDS REVIEW
- Fault gate: 13 broken builds; 13 passing tests never failed → NEEDS REVIEW
- Wrong tests among FAILs: TC-10.1, TC-11.1, TC-20.1, TC-27.1, TC-27.2, TC-28.1
- Total run time 23 min (gpt-5.6-luna)

Backup · Threats to validity and mitigations

Threat	In the demos	Mitigation in the PhD
Construct	Wrong test = still fails after the planted defects are fixed; planted defects approximate real faults	Expert-validated oracles; real fault data from industrial partners
Internal	Ground truth and prompts designed by me	Independent labelling; several prompt variants; pre-registered designs
External	One web shop, one AI model, 62 tests in one run	Open-source ACPS, industrial cases, InnoGuard platform, open-weight models
Conclusion	6 wrong tests out of 18 FAILs: small numbers	Repeated runs, larger benchmarks, effect sizes and confidence intervals

Backup · Towards uncertainty of quantum ML models (O4)

- **Starting point:** QML is already used for test oracles of autonomous mobile robots and QoS prediction of elevator software [20, 21]
- **Sources of uncertainty:** shot noise, hardware noise and model (epistemic) uncertainty add to data uncertainty
- **Transfer test:** do sampling-based metrics still predict wrong oracle verdicts when the model is quantum? Repeated shots are natural samples
- **Coverage & mutation for quantum programs** as ground-truth tools [19]
- **Road map:** quantum AI for software engineering [22]



[19] S. Ali et al., “Assessing the effectiveness of input and output coverage criteria for testing quantum programs,” Proc. IEEE ICST, 2021.
[20] X. Wang et al., “Quantum machine learning-based test oracle for autonomous mobile robots,” Proc. IEEE/ACM ASE, 2025.
[21] X. Wang et al., “Application of quantum extreme learning machines for QoS prediction of elevators' software in an industrial context,” Companion Proc. ACM FSE, 2024.
[22] X. Wang et al., “Quantum artificial intelligence for software engineering: The road ahead,” ACM TOSEM, 2026.

References (1/2)

[1] M. Zhang, S. Ali, and T. Yue, "Uncertainty-wise test case generation and minimization for cyber-physical systems," J. Syst. Softw., vol. 153, pp. 1–21, 2019, doi: 10.1016/j.jss.2019.03.011.

[2] M. Zhang, S. Ali, T. Yue, R. Norgren, and O. Okariz, "Uncertainty-wise cyber-physical system test modeling," Softw. Syst. Model., vol. 18, no. 2, pp. 1379–1418, 2019, doi: 10.1007/s10270-017-0609-6.

[3] M. Zhang, B. Selic, S. Ali, T. Yue, O. Okariz, and R. Norgren, "Understanding uncertainty in cyber-physical systems: A conceptual model," in Proc. ECMFA, LNCS 9764, 2016, pp. 247–264, doi: 10.1007/978-3-319-42061-5_16.

[4] F. O. Catak, T. Yue, and S. Ali, "Uncertainty-aware prediction validator in deep learning models for cyber-physical system data," ACM TOSEM, vol. 31, no. 4, pp. 1–31, 2022, doi: 10.1145/3527451.

[5] L. Han, S. Ali, T. Yue, A. Arrieta, and M. Arratibel, "Uncertainty-aware robustness assessment of industrial elevator systems," ACM TOSEM, vol. 32, no. 4, pp. 1–51, 2023, doi: 10.1145/3576041.

[6] P. Valle, A. Arrieta, L. Han, S. Ali, and T. Yue, "Defining and generating multi-level and uncertainty-wise test oracles for cyber-physical systems," Softw. Syst. Model., vol. 24, no. 3, pp. 679–704, 2025, doi: 10.1007/s10270-025-01271-8.

[7] P. Valle, C. Lu, S. Ali, and A. Arrieta, "Evaluating uncertainty and quality of vision-language-action-enabled robots," arXiv:2507.17049, 2025.

[8] J. Wu, C. Lu, A. Arrieta, T. Yue, and S. Ali, "Reality bites: Assessing the realism of driving scenarios with large language models," in Proc. IEEE/ACM FORGE, 2024, pp. 40–51, doi: 10.1145/3650105.3652296.

[9] H. Sartaj, J. Boudjadar, M. Frasheri, S. Ali, and P. G. Larsen, "Identifying uncertainty in self-adaptive robotics with large language models," IEEE Softw., vol. 43, no. 1, pp. 89–97, 2026, doi: 10.1109/MS.2025.3620578.

[10] C. Lu, P. Valle, J. Wu, E. Isaku, H. Sartaj, A. Arrieta, and S. Ali, "Foundation models for software engineering of cyber-physical systems: The road ahead," ACM TOSEM, vol. 35, no. 9, pp. 1–58, 2026, doi: 10.1145/3830403.

[11] S. Ali, P. Arcaini, and A. Arrieta, "Foundation models for the digital twins creation of cyber-physical systems," in Proc. ISoLA 2024, LNCS, Springer, 2025, pp. 9–26, doi: 10.1007/978-3-031-75390-9_2.

[12] A. Calò, P. Arcaini, S. Ali, F. Hauer, and F. Ishikawa, "Generating avoidable collision scenarios for testing autonomous driving systems," in Proc. IEEE ICST, 2020, pp. 375–386, doi: 10.1109/ICST46399.2020.00045.

[13] T. Laurent, S. Klikovits, P. Arcaini, F. Ishikawa, and A. Ventresque, "Parameter coverage for testing of autonomous driving systems under uncertainty," ACM TOSEM, vol. 32, no. 3, pp. 1–31, 2023, doi: 10.1145/3550270.

[14] Z. Zhang, D. Lyu, P. Arcaini, L. Ma, I. Hasuo, and J. Zhao, "FalsifAI: Falsification of AI-enabled hybrid control systems guided by time-aware coverage criteria," IEEE TSE, vol. 49, no. 4, pp. 1842–1859, 2023, doi: 10.1109/TSE.2022.3194640.

[15] Z. Zhang, D. Lyu, P. Arcaini, L. Ma, I. Hasuo, and J. Zhao, "Effective hybrid system falsification using Monte Carlo tree search guided by QB-robustness," in Proc. CAV, LNCS, 2021, pp. 595–618, doi: 10.1007/978-3-030-81685-8_29.

[16] A. Arrieta, J. Ayerdi, M. Illarramendi, A. Agirre, G. Sagardui, and M. Arratibel, "Using machine learning to build test oracles: An industrial case study on elevators dispatching algorithms," in Proc. IEEE/ACM AST, 2021, pp. 30–39, doi: 10.1109/AST52587.2021.00012.

References (2/2)

[17] J. Ayerdi, V. Terragni, A. Arrieta, P. Tonella, G. Sagardui, and M. Arratibel, "Generating metamorphic relations for cyber-physical systems with genetic programming: An industrial case study," in Proc. ESEC/FSE, 2021, pp. 1264–1274, doi: 10.1145/3468264.3473920.

[18] A. Arrieta, P. Valle, J. A. Agirre, and G. Sagardui, "Some seeds are strong: Seeding strategies for search-based test case selection," ACM TOSEM, vol. 32, no. 1, pp. 1–47, 2023, doi: 10.1145/3532182.

[19] S. Ali, P. Arcaini, X. Wang, and T. Yue, "Assessing the effectiveness of input and output coverage criteria for testing quantum programs," in Proc. IEEE ICST, 2021, pp. 13–23, doi: 10.1109/ICST49551.2021.00014.

[20] X. Wang, Q. Xu, P. Arcaini, S. Ali, and T. Peyrucain, "Quantum machine learning-based test oracle for autonomous mobile robots," in Proc. IEEE/ACM ASE, 2025, pp. 3695–3706, doi: 10.1109/ASE63991.2025.00306.

[21] X. Wang, S. Ali, A. Arrieta, P. Arcaini, and M. Arratibel, "Application of quantum extreme learning machines for QoS prediction of elevators' software in an industrial context," in Companion Proc. ACM FSE, 2024, pp. 399–410, doi: 10.1145/3663529.3663859.

[22] X. Wang, S. Ali, and P. Arcaini, "Quantum artificial intelligence for software engineering: The road ahead," ACM TOSEM, vol. 35, no. 9, pp. 1–42, 2026, doi: 10.1145/3827610.

[23] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in Proc. ICLR, 2023. arXiv:2302.09664.

[24] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy," Nature, vol. 630, no. 8017, pp. 625–630, 2024, doi: 10.1038/s41586-024-07421-0.

[25] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in Proc. EMNLP, 2023, pp. 9004–9017, doi: 10.18653/v1/2023.emnlp-main.557.

[26] Y. Huang, J. Song, Z. Wang, S. Zhao, H. Chen, F. Juefei-Xu, and L. Ma, "Look before you leap: An exploratory study of uncertainty analysis for large language models," IEEE TSE, vol. 51, no. 2, pp. 413–429, 2025, doi: 10.1109/TSE.2024.3519464.

[27] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," IEEE TSE, vol. 50, no. 4, pp. 911–936, 2024, doi: 10.1109/TSE.2024.3368208.

[28] M. Weiss and P. Tonella, "Fail-safe execution of deep learning based systems through uncertainty monitoring," in Proc. IEEE ICST, 2021, pp. 24–35, doi: 10.1109/ICST49551.2021.00015.

[29] E. Leurent, "An environment for autonomous driving decision-making," GitHub repository, 2018. https://github.com/Farama-Foundation/HighwayEnv

[30] InnoGuard: Hybrid and Generative Intelligence for Trustworthy Autonomous Cyber-Physical Systems, Horizon Europe MSCA Doctoral Network, GA No. 101169233, 2024–2028. https://cordis.europa.eu/project/id/101169233

[31] Z. V. Le, S. Hong, and Y. Choi, "Performance of test generation techniques on property checking: A case study," Korea Congress of Computing (KCC), 2025.

[32] Z. V. Le, S. Hong, and Y. Choi, "Stateful system, stateless techniques: Initial assessment on test generation for property checking of controller software," Korea Conference on Software Engineering (KCSE), 2026.